

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	Aim: Simulate Continuous & Discrete time Signals	
Experiment No: 01	Date:10-07-2026	Enrollment No:92400133121

1.

Aim: To plot unit impulse signal using Python.

Program:

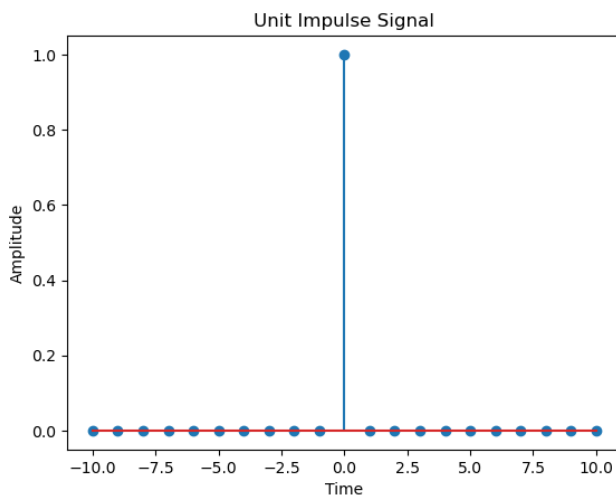
```
import numpy as np
import matplotlib.pyplot as plt
def unit_impulse(length, position):
    signal = np.zeros(length)
    signal[position] = 1
    return signal

start = -10 # Start value
stop = 10 # Stop value
step = 1 # Step size

x = np.arange(start, stop+step, step)
impulse_signal = unit_impulse(len(x), abs(start)//step)

plt.stem(x, impulse_signal)
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.title('Unit Impulse Signal')
plt.show()
```

Result:



Conclusion:

The unit impulse signal was generated using python where amplitude is 1 at a single instance($t=0$) and 0 elsewhere

2.

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	Aim: Simulate Continuous & Discrete time Signals	
Experiment No: 01	Date:10-07-2026	Enrollment No:92400133121

Aim: Simulate impulse train

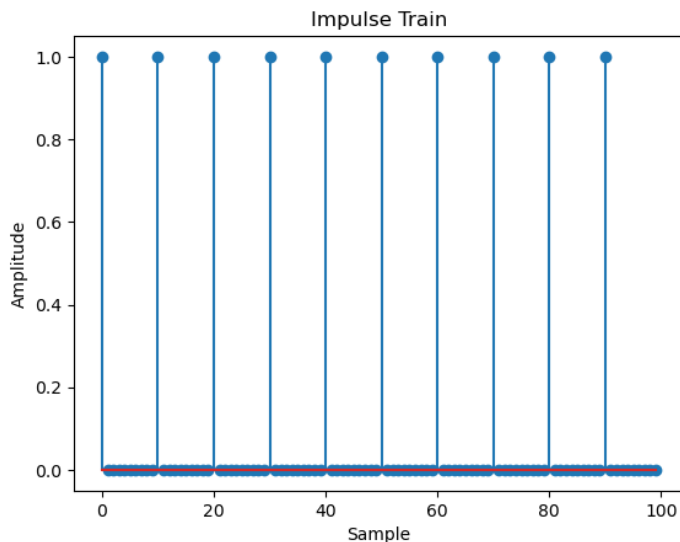
Program:

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_impulse_train(signal_length, period):
    impulse_train = np.zeros(signal_length)
    for n in range(signal_length):
        if n % period == 0:
            impulse_train[n] = 1
    return impulse_train

signal_length = 100 # Length
period = 10 # Period
impulse_train = simulate_impulse_train(signal_length, period)

plt.stem(impulse_train)
plt.title('Impulse Train')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.show()
```

Result:



Conclusion:

Impulse train was generated using python which shows the periodic nature of impulse train

3.
Aim: Write a python program to Simulate continuous and discrete unit step signal,

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	Aim: Simulate Continuous & Discrete time Signals	
Experiment No: 01	Date:10-07-2026	Enrollment No:92400133121

Program:

```

import numpy as np
import matplotlib.pyplot as plt

def simulate_continuous_unit_step(time):
    unit_step = np.zeros_like(time)
    unit_step[time >= 0] = 1
    return unit_step
def simulate_discrete_unit_step(num_samples):
    unit_step = np.zeros(num_samples)
    unit_step[num_samples // 2:] = 1
    return unit_step

time = np.linspace(-5, 5, 1000) # Time range
continuous_unit_step = simulate_continuous_unit_step(time)

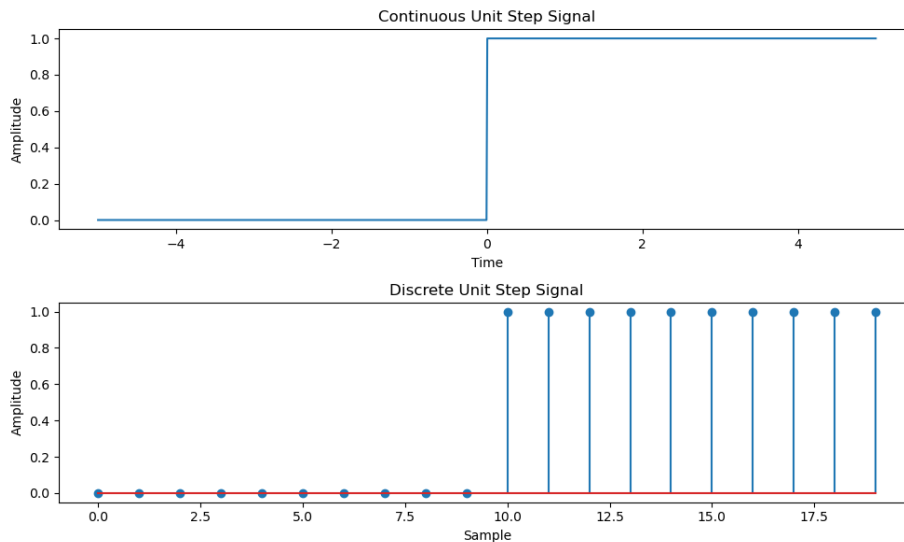
num_samples = 20 # Number of samples
discrete_unit_step = simulate_discrete_unit_step(num_samples)

plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_unit_step)
plt.title('Continuous Unit Step Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_unit_step)
plt.title('Discrete Unit Step Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()

```

Result:

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	Aim: Simulate Continuous & Discrete time Signals	
Experiment No: 01	Date:10-07-2026	Enrollment No:92400133121



Conclusion

Unit Step signal in two different domains was successfully generated using python which is 1 for ($n > 0$) or ($t > 0$).

4.

Aim: Write a python program to Simulate continuous and discrete ramp signal,

Program:

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_continuous_ramp(time, slope):
    ramp = np.zeros_like(time)
    ramp[time >= 0] = slope * time[time >= 0]
    return ramp
def simulate_discrete_ramp(num_samples, slope):
    ramp = np.zeros(num_samples)
    ramp[num_samples // 2:] = slope * np.arange(num_samples // 2, num_samples)
    return ramp

time = np.linspace(-5, 5, 1000) # Time range

num_samples = 20 # Number of samples
slope = 2 # Slope of the ramp

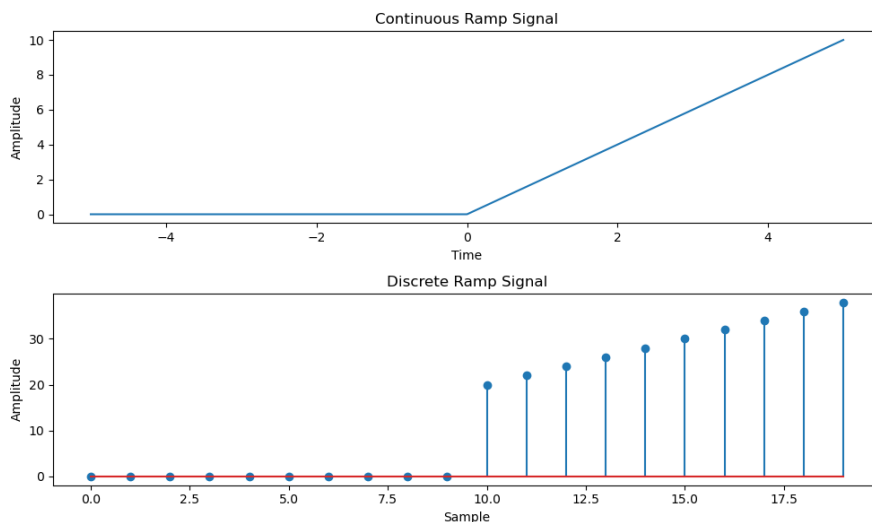
continuous_ramp = simulate_continuous_ramp(time, slope)
discrete_ramp = simulate_discrete_ramp(num_samples, slope)

plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	Aim: Simulate Continuous & Discrete time Signals	
Experiment No: 01	Date:10-07-2026	Enrollment No:92400133121

```
plt.plot(time, continuous_ramp)
plt.title('Continuous Ramp Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_ramp)
plt.title('Discrete Ramp Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()
```

Result:



Conclusion

Ramp signal in two different domains was generated successfully using python where amplitude is a for ($n > a$) or ($t > a$)

5. Aim: Write a python program to Simulate continuous and discrete exponential signal

Program:

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_continuous_exponential(time, amplitude, coefficient):
    exponential_signal = amplitude * np.exp(coefficient * time)
    return exponential_signal
def simulate_discrete_exponential(num_samples, amplitude, coefficient):
    exponential_signal = amplitude * np.exp(coefficient * np.arange(num_samples))
    return exponential_signal
```

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	Aim: Simulate Continuous & Discrete time Signals	
Experiment No: 01	Date:10-07-2026	Enrollment No:92400133121

```

time = np.linspace(0, 5, 1000) # Time range

num_samples = 20 # Number of samples
amplitude = 3 # Initial amplitude
coefficient = 1 # Exponential coefficient

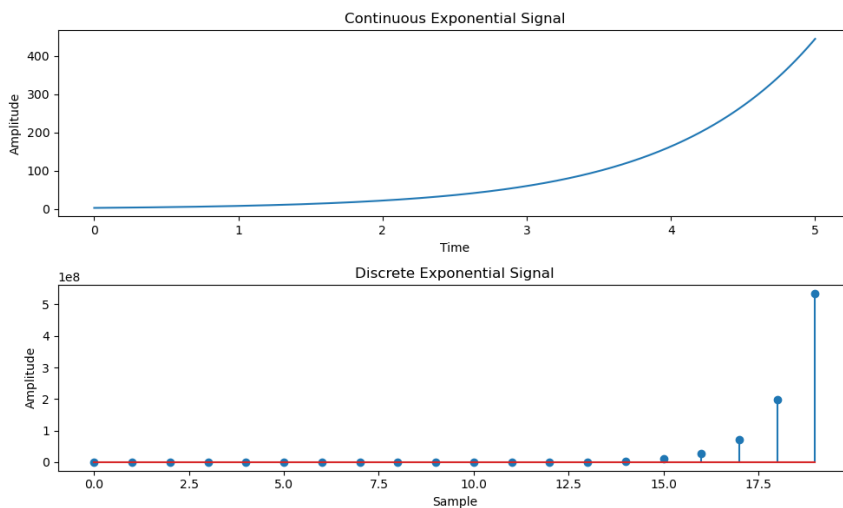
continuous_exponential = simulate_continuous_exponential(time, amplitude, coefficient)

discrete_exponential = simulate_discrete_exponential(num_samples, amplitude, coefficient)

plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_exponential)
plt.title('Continuous Exponential Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_exponential)
plt.title('Discrete Exponential Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()

```

Result:



Conclusion

Exponential signals in two different domains were generated successfully using python where the exponential growth is observed with time.

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology
Subject: DSIP (01CT1513)	Aim: Simulate Continuous & Discrete time Signals
Experiment No: 01	Date:10-07-2026

Aim: Write a python program to Simulate continuous and discrete parabolic signal

Program:

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_continuous_parabolic(time, coefficients):
    parabolic_signal = np.polyval(coefficients, time)
    return parabolic_signal
def simulate_discrete_parabolic(num_samples, coefficients):
    parabolic_signal = np.polyval(coefficients, np.arange(num_samples))
    return parabolic_signal

time = np.linspace(-5, 5, 1000) # Time range

num_samples = 20 # Number of samples
coefficients = [1, 2, 1] # Coefficients

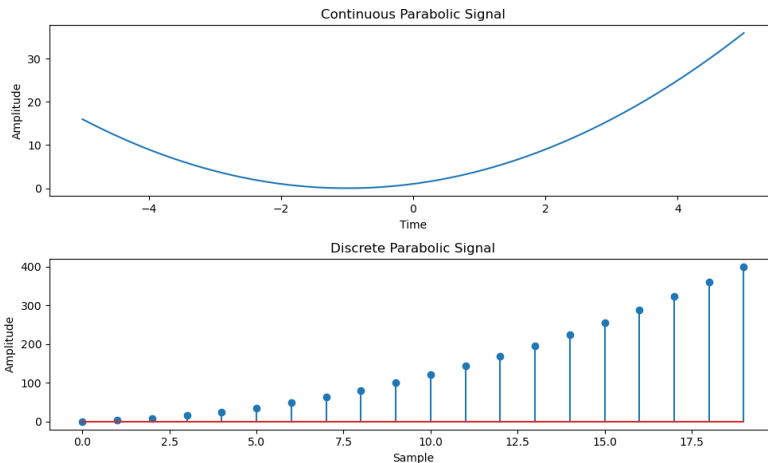
continuous_parabolic = simulate_continuous_parabolic(time, coefficients)

discrete_parabolic = simulate_discrete_parabolic(num_samples, coefficients)

plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_parabolic)
plt.title('Continuous Parabolic Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_parabolic)
plt.title('Discrete Parabolic Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()
```

Result:

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology
Subject: DSIP (01CT1513)	Aim: Simulate Continuous & Discrete time Signals
Experiment No: 01	Date:10-07-2026
	Enrollment No:92400133121



Conclusion

Parabolic signal was generated successfully in two different domains using python where it shows the quadratic behaviour of amplitude with time

7. Aim: Write a python program to Simulate continuous and discrete sine wave signal

Program:

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_continuous_sine_wave(time, amplitude, frequency, phase):
    sine_wave = amplitude * np.sin(2 * np.pi * frequency * time + phase)
    return sine_wave
def simulate_discrete_sine_wave(num_samples, sampling_frequency, amplitude, frequency, phase):
    time = np.arange(num_samples) / sampling_frequency
    sine_wave = amplitude * np.sin(2 * np.pi * frequency * time + phase)
    return sine_wave
```

```
time = np.linspace(0, 1, 1000) # Time range
```

```
num_samples = 50 # Number of samples
sampling_frequency = 5 # Sampling frequency (Hz)
amplitude = 1 # Amplitude
frequency = 2 # Frequency (Hz)
phase = 0 # Phase angle(radians)
```

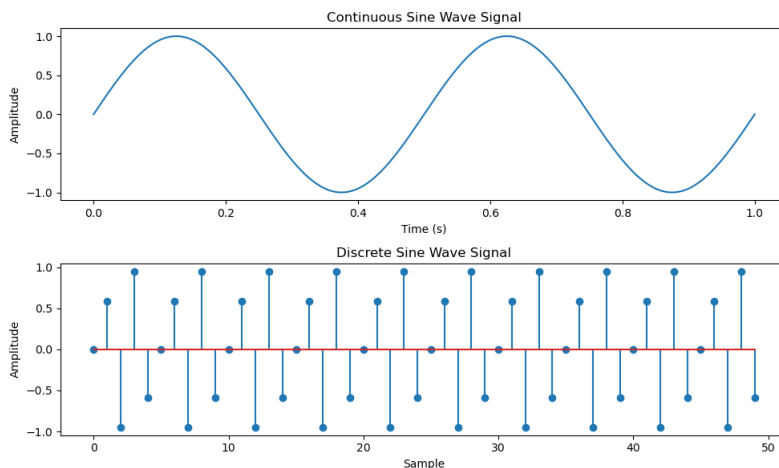
```
continuous_sine_wave = simulate_continuous_sine_wave(time, amplitude, frequency, phase)
```

```
discrete_sine_wave = simulate_discrete_sine_wave(num_samples, sampling_frequency,
amplitude, frequency, phase)
```


 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	Aim: Simulate Continuous & Discrete time Signals	
Experiment No: 01	Date:10-07-2026	Enrollment No:92400133121

```
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_sine_wave)
plt.title('Continuous Sine Wave Signal')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_sine_wave)
plt.title('Discrete Sine Wave Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()
```

Result:



Conclusion

Sine wave in two different domains is generated successfully using python where the periodic nature is observed.

8

Aim: Write a python program to simulate $y(t)=u(t)+u(t-1)+3u(t+5)$

Program:

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_function(time):
    y = np.zeros_like(time)
    y[time >= 0] = 1
    y[time >= 1] += 1
    y[time >= -5] += 3
```

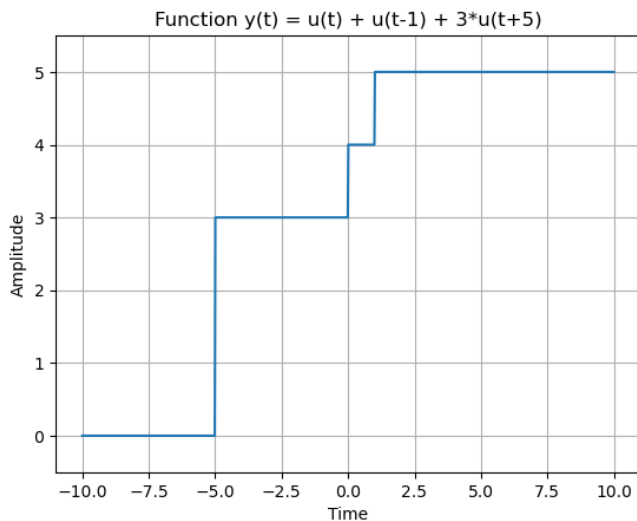
 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	Aim: Simulate Continuous & Discrete time Signals	
Experiment No: 01	Date:10-07-2026	Enrollment No:92400133121

```
# 1,1,3 - coefficients
return y
```

```
time = np.linspace(-10, 10, 1000) //Time range
function_values = simulate_function(time)
```

```
plt.plot(time, function_values)
plt.title('Function y(t) = u(t) + u(t-1) + 3*u(t+5)')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.ylim([-0.5, 5.5])
plt.grid(True)
plt.show()
```

Result:



Conclusion:

A complex signal can be generated combining unit step signals where time shifting and scaling property is being observed and plotted using python.

9.

Aim: Write a python program to simulate $y(t)=\Delta(t)+\delta(t-1)+3*\delta(t+5)$.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_function(time):
    y = np.zeros_like(time)
    y[time == 0] = 1
```

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	Aim: Simulate Continuous & Discrete time Signals	
Experiment No: 01	Date:10-07-2026	Enrollment No:92400133121

```

y[time == -1] += 3
y[time == 5] += 4
# 1,3,4 - coefficients
return y

```

```

time = np.arange(-10, 11) //Time range
function_values = simulate_function(time)

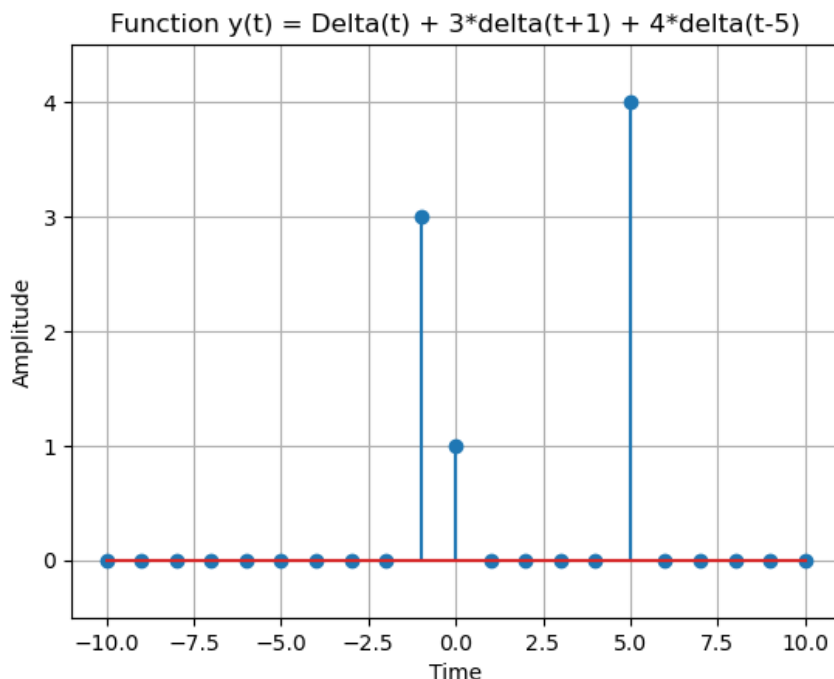
```

```

plt.stem(time, function_values)
plt.title('Function y(t) = Delta(t) + 3*delta(t+1) + 4*delta(t-5)')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.ylim([-0.5, 4.5])
plt.grid(True)
plt.show()

```

Result:



Conclusion:

A complex signal can be generated combining unit impulse signals where time shifting and scaling property is being observed and plotted using python.